ByteTrack Under Camera Motion: Reproduction and Error Analysis on MOT17

Vanodhya Warnasooriya
Department of Electrical Engineering, Chulalongkorn University, Bangkok, Thailand
Student ID 6872086821 2102515 Digital Video Processing, Semester 1/2026

***Abstract*—ByteTrack predicts every track with a constant-velocity Kalman filter in image coordinates, so a moving camera appears to the filter as motion of every person. We reproduce ByteTrack on the MOT17 half-validation split with the official code and weights and obtain 76.6 MOTA and 79.3 IDF1, equal to the published values. We then split the error by camera type. The four moving-camera sequences hold 31% of the ground-truth boxes but 48% of the identity switches, and their HOTA is 10.9 points lower than on static cameras, a gap that stays between 9.8 and 12.6 points under six tracker and detector settings. The loss is larger in association (AssA −13.4) than in detection (DetA −8.4). A per-frame estimate of global motion (ORB features and a RANSAC fit) shows that camera motion degrades the one-frame Kalman prediction, but the predicted box still overlaps its detection well enough to be matched. The penalty appears when a lost track must be re-found: after 9–16 lost frames, 2.1% of lost tracks are recovered with a moving camera against 6.0% with a static one.**

***Index Terms*—multi-object tracking, ByteTrack, Kalman filter, camera motion, reproducibility, MOT17**

I. Introduction

Tracking-by-detection links the boxes of a detector over time. ByteTrack [1] is a strong and simple example: a YOLOX detector [2] gives boxes, a Kalman filter [3] predicts where each track will be in the next frame, and the boxes are matched to the predictions by intersection over union (IoU). Its key idea, BYTE, also matches low-score boxes in a second step, which recovers people who are partly occluded.

The Kalman filter assumes constant velocity *in image coordinates*. When the camera pans, walks or drives, every person moves in the image even if they stand still. About half of the MOT17 sequences are filmed with a moving camera [4], but the ByteTrack paper reports only results averaged over all sequences. It is therefore not known how much of ByteTrack's error comes from camera motion, and through which part of the tracker.

This Level A study reproduces ByteTrack and analyses its error by camera type. Our contributions are:

- a reproduction of ByteTrack on MOT17 half-val that matches the published MOTA and IDF1, together with an explanation of the setting behind the published numbers (Sec. IV-A);
- a split of the error into static- and moving-camera sequences, which shows a stable gap of about 10.9 HOTA points, mainly in association (Sec. IV-B);
- a per-frame mechanism study that relates measured camera motion to the Kalman prediction and locates the loss in the recovery of lost tracks (Sec. IV-C), and a sensitivity study showing that the gap persists across tracker settings (Sec. IV-D).

II. Related Work

Motion-based association. SORT [5] introduced the Kalman filter plus IoU and Hungarian matching that ByteTrack still uses; its motion model is explicitly independent of camera motion. DeepSORT [6] added appearance features and the 8-D box state that ByteTrack inherits. ByteTrack [1] keeps the motion model and changes which detections are associated; it reports that the Kalman filter fails on BDD100K because of large camera motion, but gives no per-camera breakdown on MOT17. **Camera motion compensation.** BoT-SORT [7] adds global motion compensation to ByteTrack and improves IDF1 by 1.6 points on MOT17 half-val, but it reports only overall numbers and does not show where the gain comes from. OC-SORT [8] shows that Kalman errors accumulate while a track has no observation, but it studies occlusion and non-linear motion, not the camera. **Metrics.** MOTA [9] is dominated by detection errors and IDF1 [10] by identity errors; HOTA [11] separates detection (DetA) from association (AssA), which we use to locate the loss.

III. Experimental Setup

Data and split. We use the seven MOT17 training sequences (FRCNN copies) and the half split of ByteTrack [1]: the first half of each sequence was used for training, the second half (2,652 frames) for evaluation. Following the MOT16/17 benchmark table [4], MOT17-02, 04 and 09 are static-camera sequences and MOT17-05, 10, 11 and 13 are moving-camera sequences.

Baseline. We run the official ByteTrack code (commit `d1bf019`) with its `ByteTrack_ablation` weights (YOLOX-X trained on CrowdHuman and the first half of MOT17), input 800×1440, FP16 inference, and the default tracker settings of `tools/track.py`: high-score threshold 0.6, matching threshold 0.9, and a lost-track buffer of 30 frames (14 for MOT17-05 and 25 for MOT17-13, as hard-coded in the official evaluator). The tracker code is unchanged; a small compatibility layer only replaces a compiled IoU routine with an equivalent NumPy version. Detections are computed once and cached, so every tracker variant uses identical boxes.

TABLE I
E1: REPRODUCTION ON MOT17 HALF-VAL (*: BYTETRACK'S OWN EVALUATION).

Setting	MOTA*	IDF1*	IDs*	HOTA	DetA	AssA
Paper [1]	76.6	79.3	159	–	–	–
Official settings	76.6	79.3	164	67.9	66.6	69.8
+ interpolation (20)	78.3	80.2	148	69.0	68.0	70.6
Match thr. 0.8	76.4	79.0	202	67.9	66.4	69.9
Match 0.8 + interp.	78.0	79.8	187	68.9	67.7	70.8
Detector in FP32	76.6	78.7	167	67.6	66.7	69.0
SORT, same detections	74.6	77.2	287	66.5	65.0	68.5

Metrics. We report MOTA, IDF1 and ID switches with ByteTrack's own evaluation (py-motmetrics, marked *) for a direct comparison with the paper, and HOTA, DetA and AssA with TrackEval [11].

Camera motion. For each frame pair we detect up to 3,000 ORB features [12] on a 960-pixel-wide grey image, mask out the detector's own person boxes, match the features with a ratio test, and fit a similarity transform A_t (rotation, scale, translation) with RANSAC [13]. Camera motion is the mean displacement of the four image corners under A_t, in percent of the image diagonal per frame. The fit succeeds in at least 92% of the frames of every sequence.

Prediction log. We record, without changing any result, every Kalman-predicted box at BYTE's first association step, the detection it is matched to and the IoU between the two. Re-running the tracker with the log produces identical tracks.

IV. EXPERIMENTS

A. E1: Reproduction

Table I lists the reproduction and five variants. With the official settings we obtain 76.6 MOTA, 79.3 IDF1 and 164 ID switches, against 76.6, 79.3 and 159 in the paper. Adding the paper's linear tracklet interpolation (interval 20) gives 78.3 MOTA and 80.2 IDF1, again equal to the paper's Table 11, and our HOTA of 67.9 matches the 67.88 that BoT-SORT [7] reports for ByteTrack on the same split. ByteTrack's SORT on the same detections gives 74.6 MOTA (paper: 74.6).

The paper's text rejects matches below an IoU of 0.2, which corresponds to a matching threshold of 0.8, while the code uses 0.9. With 0.8 we obtain 76.4 MOTA and 202 ID switches, so the published numbers were produced with the code default. Running the detector in FP32 instead of FP16 changes IDF1 by less than one point. The remaining difference of five ID switches is within what we would expect from different GPU and library versions.

B. E2: Static vs. moving camera

Table II splits the official run by sequence. Static cameras reach 71.2 HOTA and moving cameras 60.3. The drop is larger in association (AssA 73.5 vs. 60.1) than in detection (DetA 69.4 vs. 60.9). Moving-camera sequences contain 31% of the ground-truth boxes but 48% of the identity switches, i.e. 4.8 instead of 2.4 switches per 1,000 boxes. The gap is stable: under all six settings of Table I it lies between 9.8 and 12.6 HOTA points, and the AssA gap is always larger than the DetA gap.

The two groups differ in more than camera motion. However, the differences we can measure favour the moving group: its sequences are less crowded, its people are larger in the image and better visible (Table II). Crowding therefore cannot explain the gap, which motivates a per-frame analysis inside each sequence.

C. E3: Camera motion and the Kalman prediction

Camera motion. The median camera motion is at most 0.003% of the diagonal per frame on static sequences, which is estimation noise, and 0.18–0.70% on moving sequences, about one hundred times more.

One-frame prediction. Inside every moving sequence, frames with more camera motion have a lower prediction IoU (Spearman ρ from -0.43 to -0.25, $p < 0.001$; Fig. 1, left). At the level of single matches ($n = 44{,}507$), the centre error of the prediction grows with the image motion that the camera causes at the box ($\rho = 0.51$): from 0.011 to 0.047 box heights between the lowest and highest 10% of camera flow. The mechanism of Sec. I is therefore real. Yet the error stays small compared with the matching gate: a prediction IoU around 0.83 is far above the 0.1 that a match needs. Accordingly, when we split each moving sequence at its median camera motion, the high-motion half has 4.98 identity switches per 1,000 boxes and the low-motion half 4.61 (41 vs. 39 events), which is no clear difference.

Lost tracks. The loss appears when a track has been lost and must be found again (Fig. 2). The prediction of a lost track is extrapolated over several frames, and its best IoU with any detection falls much faster with a moving camera. After 9–16 lost frames, the best IoU is 0.10 against 0.27, and only 2.1% of the lost tracks are re-found, against 6.0% on static cameras. A track that is not re-found comes back with a new identity, which is consistent with the larger AssA gap in E2. Fig. 3 shows a frame of MOT17-13 with the predicted and detected boxes.

Within the moving sequences, however, the camera motion accumulated during the gap does not predict recovery once the gap length is fixed (median $\rho = 0.00$ over 30 gap lengths; pooled $\rho = -0.15$ is driven by gap length). The penalty is therefore a property of moving-camera video as a whole rather than proportional to the size of the motion.

D. E4: Sensitivity study

We change one tracker parameter at a time around the official settings: the high-score threshold (0.4–0.7), the matching threshold (0.7–0.95), the lost-track buffer (10–90 frames) and the standard deviation of the Kalman *process* noise ($\times 0.25$ to $\times 4$, measurement noise fixed). Scaling all noise terms of ByteTrack's filter together leaves the Kalman gain, and therefore every prediction, unchanged, so only the process noise is varied. The aim is to test whether the gap is stable, not to remove it.

TABLE II

PER-SEQUENCE PROPERTIES AND RESULTS (OFFICIAL SETTINGS). MOTION: MEDIAN CAMERA MOTION PER FRAME (E3). PRED. IOU: MEAN IOU BETWEEN A ONE-FRAME KALMAN PREDICTION AND ITS MATCHED DETECTION. ρ: SPEARMAN CORRELATION OVER THE FRAMES OF THE SEQUENCE.

Seq.	Cam.	FPS	People/ frame	Height (%)	Scene	HOTA	DetA	AssA	IDSW/ 1k GT	Motion (%diag)	Pred. IoU	ρ(motion, pred. IoU)
02	S	30	33.0	9	cloudy	47.6	49.0	46.9	5.8	0.003	0.89	-0.22
04	S	30	46.1	16	night	79.7	78.1	81.6	0.7	0.002	0.95	-0.12
09	S	30	11.0	25	indoor	65.5	70.2	61.1	4.9	0.003	0.89	-0.20
05	M	14	8.0	50	sunny	60.9	63.9	58.4	8.3	0.702	0.83	-0.31
10	M	30	18.2	10	night	54.9	56.5	53.9	4.4	0.182	0.85	-0.43
11	M	30	10.1	41	indoor	62.3	63.9	61.0	2.4	0.236	0.90	-0.43
13	M	25	8.4	11	sunny	64.8	62.6	67.3	5.1	0.650	0.81	-0.25
All static						71.2	69.4	73.5	2.4			
All moving						60.3	60.9	60.1	4.8			

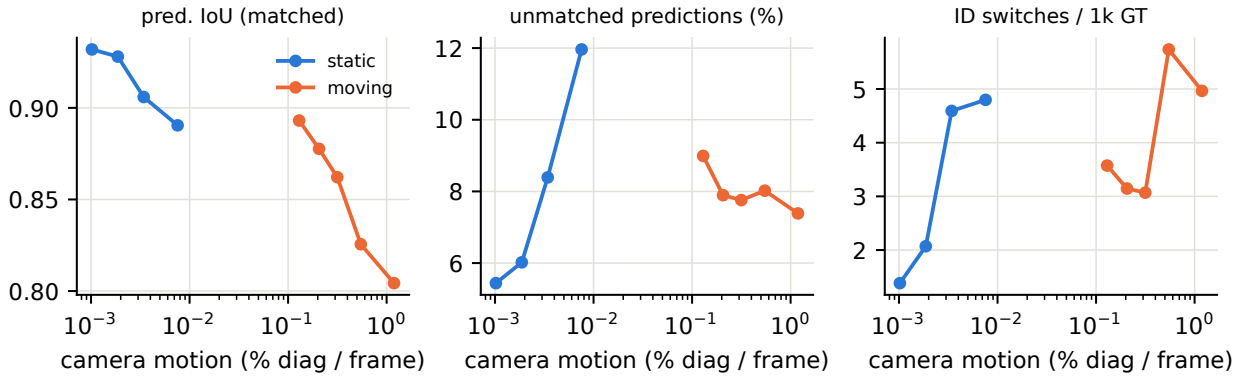


Fig. 1. Frame-level relations, binned by camera motion. Left: IoU of one-frame predictions with their matched detections. Middle: share of predictions without a matching high-score detection. Right: ID switches per 1,000 ground-truth boxes. Bins mix sequences; claims in the text use within-sequence statistics.

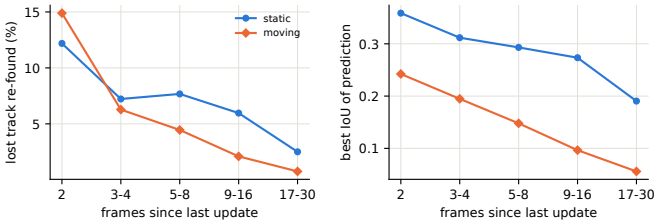


Fig. 2. Recovery of lost tracks. Left: share of lost-track predictions matched again. Right: best IoU between the prediction and any high-score detection.

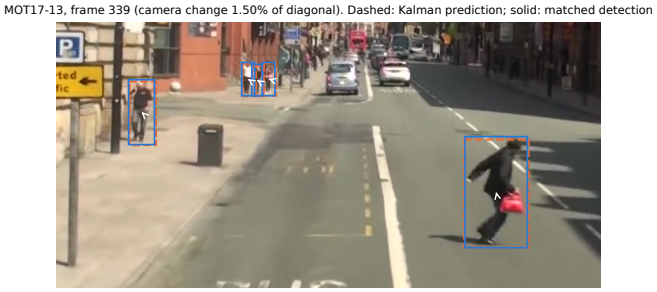


Fig. 3. MOT17-13 (moving camera). Dashed: Kalman predictions; solid: matched detections; arrows: correction.

Across all 15 settings, moving cameras stay between 8.8 and 16.5 HOTA points behind static cameras (Fig. 4), and the AssA gap exceeds the DetA gap in 14 of them. The thresholds and the buffer hardly move the moving-camera result (HOTA 59.2–61.0; gap 9.8–11.7). The Kalman process noise is the only parameter that acts mainly on the moving group: making the filter trust its constant-velocity model more ($\times 0.25$) lowers moving-camera HOTA to 54.2 while static cameras keep 70.8, and allowing more process noise ($\times 2$) raises it to 61.5 while static cameras drop to 70.3. This asymmetry is consistent with E3: moving-camera tracks depend on how quickly the filter accepts a change of image motion. Overall HOTA is highest at the official setting, so no setting removes the gap without a cost elsewhere.

V. DISCUSSION AND LIMITATIONS

Our results agree with the motivation of camera-motion compensation in BoT-SORT [7], but they locate the problem more precisely: for ByteTrack on MOT17, camera motion hardly breaks frame-to-frame matching; it mainly breaks the re-association of tracks after a gap. Three limitations apply. First, there are only seven sequences, and the groups also differ in frame rate, viewpoint and lighting, so all results are correlational. Second, our camera model is a 2-D similarity

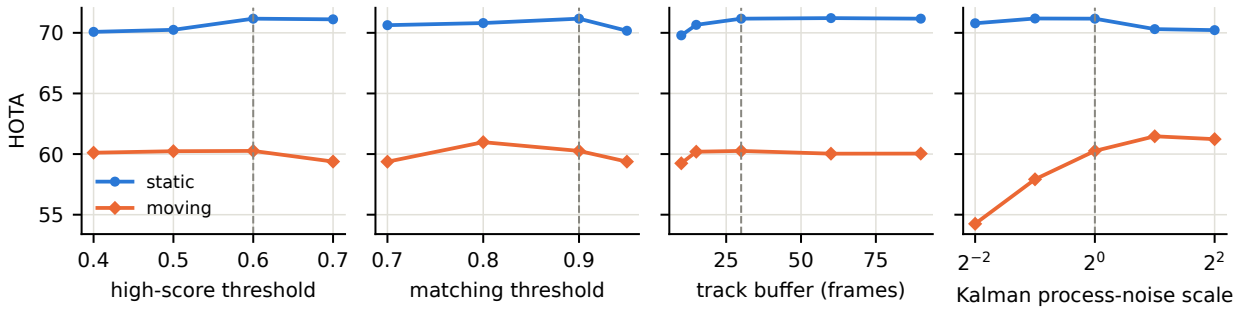


Fig. 4. E4: HOTA of static and moving cameras when one parameter changes (dashed line: official value).

transform, which cannot describe parallax or the forward motion in MOT17-13. Third, the analysis covers one detector and one tracker, and the sensitivity study changes one parameter at a time.

VI. Conclusion

We reproduced ByteTrack on MOT17 half-val with the published MOTA and IDF1 and showed that the published numbers use the code's matching threshold. Moving-camera sequences lose about 10.9 HOTA points, mainly in association. Camera motion degrades the Kalman prediction measurably, but the cost appears in the recovery of lost tracks, not in frame-to-frame matching. The gap remains under all tracker settings we tested, and only the Kalman process noise changes it noticeably, which points to the motion model, rather than the detector or the matching thresholds, as the part to examine in future work.

References

[1] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conference on Computer Vision (ECCV)*, 2022.

[2] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[3] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[4] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *arXiv preprint arXiv:1603.00831*, 2016.

[5] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016.

[6] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017.

[7] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," *arXiv preprint arXiv:2206.14651*, 2022.

[8] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[9] K. Bernardin and R. Stiefelhagen, "Evaluating multiple object tracking performance: The CLEAR MOT metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, pp. 1–10, 2008.

[10] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, "Performance measures and a data set for multi-target, multi-camera tracking," in *Proc. European Conference on Computer Vision Workshops (ECCVW)*, 2016, pp. 17–35.

[11] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A higher order metric for evaluating multi-object tracking," *International Journal of Computer Vision*, vol. 129, pp. 548–578, 2021.

[12] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2011, pp. 2564–2571.

[13] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

Appendix A
Reproducibility

Code: https://github.com/vanowarna/bytetrack-moving-camera, release `v1.0` (commit `b73239e`). **Environment:** Python 3.11, PyTorch 2.7.1 (CUDA 12.8), locked with `uv`; ByteTrack `d1bf019`; TrackEval `12c8791`; NVIDIA RTX 5050 Laptop GPU (8 GB). **Run times:** detection 58 ms per frame in FP16 (a few minutes for the 2,652 frames, run once and cached); the live demonstration on MOT17-13 (detection, tracking, evaluation and video) takes about one minute. **Seeds:** fixed; all tracking runs are deterministic given the cached detections. **Commands** (all in `scripts/`; `make_paper_assets.py` then writes every number, table and figure):

Data split	`make_splits.py`
Table I (E1)	`run_many.py --configs "configs/e1_*.yaml" --table e1`
Table II (E2)	`analyze_e2.py`
E3, Figs. 1–3	`run_camera_motion.py`, `run_e3_log.py`, `analyze_e3.py`
E4, Fig. 4	`make_e4_configs.py`, `run_many.py`, `analyze_e4.py`